

Отделение информационных технологий и вычислительной техники

КУРСОВАЯ РАБОТА

по МДК 02.01 Технология разработки программного обеспечения
специальности 09.02.07 Информационные системы и программирование

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВЕРНОЙ ЛОГИКИ (BACKEND) ДЛЯ СИСТЕМЫ УПРАВЛЕНИЯ БИЗНЕС- ПРОЦЕССАМИ СЕТЕВОЙ АВТОМАСТЕРСКОЙ

Исполнитель:
студент группы № ИП-32
_____ / Коркишко
А.Д.

Руководитель:
_____ / Прохорова И.Н.

Курсовая работа
защищена с оценкой

Новосибирск, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1. Постановка задачи	5
1.2. Анализ существующих backend решений	6
1.3. Обзор инструментальных средств.....	8
1.4. Обоснование выбора и определение методологии программы и методики испытания (ПМИ)	9
2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА BACKEND ЛОГИКИ.....	12
2.1. Анализ требований и определение спецификаций	12
2.2. Проектирование архитектуры backend.....	14
2.3. Проектирование BPMN-диаграммы бизнес-процессов	17
2.4. Реализация моделей данных.....	21
2.5 Реализация бизнес-логики	27
2.6 Тестирование серверной логики.....	30
ЗАКЛЮЧЕНИЕ	34
Список использованных источников	36
Приложение А. Листинги программного кода	37

ВВЕДЕНИЕ

Современные сетевые автомастерские функционируют в условиях высокой конкуренции и возрастающих требований клиентов к скорости и качеству обслуживания. В этих условиях эффективное управление бизнес-процессами невозможно без внедрения информационных систем, обеспечивающих автоматизацию ключевых операций: регистрации клиентов, учета транспортных средств, обработки заявок и управления персоналом.

Актуальность разработки серверной логики для системы управления автомастерской обусловлена необходимостью создания гибкого, масштабируемого и адаптируемого программного решения. Существующие коммерческие системы обладают рядом недостатков, включая высокую стоимость, сложность внедрения и ограниченную кастомизацию под конкретные бизнес-процессы.

В обновлённой версии проекта особое внимание уделено реализации ролевой модели доступа, позволяющей разграничить права пользователей системы. Введена централизованная таблица ролей, а также реализовано разделение пользователей на категории: клиенты, мастера и менеджеры. Это позволило повысить безопасность, управляемость и масштабируемость системы.

Дополнительно реализован модуль управления расписанием, обеспечивающий планирование рабочих смен сотрудников, а также функционал управления персоналом через интерфейс менеджера.

Целью данной курсовой работы является проектирование и разработка серверной логики системы управления бизнес-процессами автомастерской с поддержкой ролевой модели доступа и расширенного функционала управления.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать предметную область.
выполнить обзор существующих backend-решений.
3. Обосновать выбор технологического стека.
4. Спроектировать архитектуру системы.
еализовать модели данных с учетом существующей базы PostgreSQL.
азработать бизнес-логику обработки заявок.
еализовать систему ролей и разграничения доступа.
8. Реализовать модуль управления расписанием.
беспечить целостность данных и корректность работы системы.
ровести тестирование разработанного программного обеспечения.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. Постановка задачи

Целью разработки серверной логики является создание надежного и производительного программного обеспечения, обеспечивающего автоматизацию ключевых бизнес-процессов сетевой автомастерской. Система должна поддерживать централизованное управление информационными потоками при сохранении строгой целостности и непротиворечивости данных.

В рамках постановки задачи необходимо определить границы функциональности серверной части. Ввиду ограниченности ресурсов разработка сфокусирована на критическом модуле: регистрации клиентов, учёте транспортных средств и оформлении заявок на ремонт. Разработка складских и аналитических модулей вынесена за рамки текущего этапа, однако их потенциальное внедрение в будущем должно быть учтено в архитектуре системы.

Система должна обеспечивать:

1. централизованное хранение данных;
2. защиту от дублирования записей;
3. разграничение доступа пользователей;
4. возможность масштабирования.
5. Функциональные требования:
 - регистрация пользователей с назначением роли.
 - автоматическое создание клиента при оформлении заявки.
 - создание и обработка заявок на ремонт.
9. Проверка принадлежности автомобиля клиенту.
10. Редактирование заявок.
11. Фильтрация заявок по статусу.
12. Заполнение данных по телефону и номеру авто.
13. Управление персоналом (мастера, менеджеры).

14. Управление расписанием работы мастеров.
15. Ограничение доступа по ролям.

Нефункциональные требования:

1. безопасность хранения паролей;
2. целостность данных;
3. производительность;
4. масштабируемость.

Технические требования предусматривают использование веб-фреймворка Django, реляционной СУБД PostgreSQL и драйвера Psycopg2 для обеспечения взаимодействия между приложением и базой данных.

1.2. Анализ существующих backend решений

Для обоснованного выбора технологического стека был проведён сравнительный анализ современных фреймворков для разработки серверной логики веб-приложений. Критериями сравнения выступали: производительность, безопасность, наличие встроенного ORM, масштабируемость, сообщество и документация.

Django является высокоуровневым веб-фреймворком на языке Python, реализующим архитектурный паттерн Model-View-Template (MTV).

Преимущества:

- Встроенный ORM, обеспечивающий абстракцию от конкретной СУБД и автоматическую генерацию SQL-запросов.
- Механизмы защиты от распространённых уязвимостей (CSRF, XSS, SQL-инъекции, clickjacking).
- Встроенная административная панель, позволяющая управлять данными без написания дополнительного кода.

- Обширная документация и крупное сообщество разработчиков.
- Механизм миграций, обеспечивающий версионирование схемы базы данных.

Недостатки:

- Избыточность функционала для небольших проектов.
- Более низкая производительность по сравнению с асинхронными фреймворками.

FastAPI является современным асинхронным фреймворком, поддерживающим спецификацию OpenAPI.

Преимущества:

- Высокая производительность, сопоставимая с Node.js и Go.
- Встроенная поддержка асинхронности и WebSocket.
- Автоматическая генерация интерактивной документации API.

Недостатки:

- Отсутствие встроенного ORM (требуется интеграция с SQLAlchemy).
- Меньший размер сообщества по сравнению с Django.
- Ограниченная поддержка шаблонизации для полноценных веб-приложений.

Flask является микрофреймворком, предоставляющим минимальный набор инструментов для создания веб-приложений.

Преимущества:

- Лёгкость и минимализм.
- Высокая гибкость и возможность выбора компонентов.

Недостатки:

- Отсутствие встроенных механизмов безопасности и ORM.
- Необходимость самостоятельной интеграции множества сторонних расширений.

По результатам сравнительного анализа был сделан выбор в пользу фреймворка Django. Данное решение обусловлено следующими факторами: наличие встроенного ORM для бесшовной интеграции с существующей базой данных PostgreSQL, встроенные механизмы защиты от уязвимостей, возможность быстрой разработки благодаря встроенной административной панели, а также обширная документация и активное сообщество, что критически важно в условиях курсового проектирования. Для реализации API-эндпоинтов (автозаполнение, проверка владельца) было принято решение использовать встроенные средства Django (JsonResponse, ручная обработка GET-параметров) вместо Django REST Framework (DRF).

Это обосновано следующими факторами:

проект находится на начальной стадии и объём API минимален (всего 3 эндпоинта);

использование DRF добавило бы проекту избыточную сложность на текущем этапе.

В будущем, при расширении количества эндпоинтов, возможен рефакторинг в сторону DRF, но сейчас выбрано минимальное достаточное решение.

1.3. Обзор инструментальных средств

Выбор инструментария осуществлялся с учётом необходимости сквозной разработки серверной логики: от моделирования архитектуры до реализации и тестирования программного кода.

Django выбран в качестве основного веб-фреймворка. Данный инструмент предоставляет комплексную среду для разработки серверной логики, включая ORM для взаимодействия с базой данных, встроенную систему маршрутизации, механизмы обработки HTTP-запросов и формирования ответов, систему аутентификации и авторизации.

PostgreSQL используется в качестве системы управления базами данных. Выбор обоснован следующими характеристиками: строгое соблюдение стандартов SQL, механизм транзакций с поддержкой свойств ACID, мощная система ограничений целостности (внешние ключи, уникальность, проверочные ограничения), поддержка сложных типов данных и индексов.

Psycopg2 выступает в роли драйвера для взаимодействия приложения Django с СУБД PostgreSQL. Данная библиотека реализует протокол DB-API 2.0 и обеспечивает эффективное преобразование типов данных между Python и PostgreSQL.

DBeaver использован в качестве универсального графического клиента для работы с базой данных. Через данный интерфейс осуществлялось администрирование PostgreSQL, выполнение проверочных SQL-запросов и анализ структуры существующих таблиц.

Visual Studio Code выбран в качестве основной среды разработки. Данный инструмент обеспечивает подсветку синтаксиса, автодополнение кода, интеграцию с системой контроля версий Git и встроенный терминал для выполнения команд управления проектом.

1.4. Обоснование выбора и определение методологии программы и методики испытания (ПМИ)

Разработка серверной логики осуществлялась с применением итеративной модели, предусматривающей последовательное наращивание

функциональности и промежуточное тестирование реализованных компонентов. Принцип поэтапной верификации предполагает сначала проверку отдельных модулей обработки данных, а затем их интеграцию в общую систему.

Методология проведения испытаний базируется на ГОСТ 19.301-79 «Программа и методика испытаний», что обеспечивает структурированность проверок. В качестве основной стратегии применяется метод «белого ящика» применительно к серверной логике: корректность работы оценивается путём анализа выполнения программного кода и проверки промежуточных состояний.

Выделены следующие уровни тестирования:

модульное тестирование (unit testing) – проверка отдельных функций и методов на корректность работы в изоляции от остальных компонентов системы. Данный уровень позволяет локализовать дефекты на ранних стадиях разработки.

интеграционное тестирование – проверка взаимодействия между модулями системы, включая корректность обмена данными между приложением и базой данных, а также соблюдение ограничений целостности на уровне внешних ключей.

функциональное тестирование – проверка соответствия реализованной функциональности требованиям, определённым в техническом задании. Для каждого сценария определены четкие критерии приемки, включающие корректность обработки HTTP-запросов, формирование ответов и соблюдение бизнес-правил.

Для каждого тестового сценария фиксируются следующие параметры: идентификатор и наименование сценария, выполняемое действие (SQL-запрос или HTTP-запрос), ожидаемый результат, фактический результат и отметка о прохождении.

Формализация методики испытаний закладывает прочную основу для контроля качества разрабатываемого программного обеспечения. Разработанный план ПМИ гарантирует, что на этапе реализации ни одна архитектурная ошибка не останется незамеченной, обеспечивая высокую надёжность итогового продукта.

2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА BACKEND ЛОГИКИ

2.1. Анализ требований и определение спецификаций

Сбор требований к разрабатываемой системе осуществлялся на основе анализа предметной области, а также моделирования типовых сценариев работы сотрудников и клиентов автомастерской. Основной задачей являлось выявление ключевых бизнес-процессов, подлежащих автоматизации, и формализация требований к серверной части информационной системы.

Ключевыми бизнес-процессами, реализуемыми в системе, являются:

регистрация пользователей — процесс создания учетных записей различных типов пользователей (клиенты, мастера, менеджеры). В системе предусмотрено разграничение ролей с использованием отдельного справочника ролей, что обеспечивает гибкость управления правами доступа.

создание заявок на ремонт — процесс оформления заказов на обслуживание транспортных средств, включающий ввод данных клиента, автомобиля и описания неисправности.

управление пользователями — процесс создания сотрудников менеджером, включая мастеров с возможностью назначения административных прав (администратор-мастер).

проверка принадлежности транспортного средства — механизм предотвращения создания заявок на автомобили, принадлежащие другим клиентам.

редактирование заявок — возможность изменения описания проблемы и статуса выполнения заказа в зависимости от роли пользователя.

просмотр и фильтрация заявок — получение списка заявок с возможностью фильтрации по статусу выполнения.

управление расписанием сотрудников — создание, просмотр и публикация смен для мастеров.

втозаполнение данных — автоматическое получение информации о клиенте по номеру телефона и об автомобиле по государственному номеру.

В ходе анализа были выделены основные сущности системы:

ser (Пользователь) — содержит базовую информацию: имя, фамилия, email, пароль, роль;

2. Role (Роль) — справочник ролей (client, master, manager, admin);

lient (Клиент) — расширение пользователя, включающее контактные данные;

aster (Мастер) — профиль сотрудника с дополнительным признаком административных прав;

anager (Менеджер) — профиль сотрудника, отвечающего за управление системой;

ar (Автомобиль) — информация о транспортном средстве;

rder (Заявка) — данные о заказе на ремонт;

cheduleShift (Смена) — расписание работы сотрудников.

Между сущностями определены следующие связи:

дин пользователь имеет одну роль;

лиент связан с пользователем отношением «один к одному»;

дин пользователь может владеть несколькими автомобилями;

дин клиент может иметь несколько заявок;

дна заявка связана с одним автомобилем;

дин сотрудник может иметь множество смен.

Функциональные требования к системе включают:

втоматическое создание клиента при оформлении заявки;

редотвращение дублирования пользователей по email;

онтроль уникальности автомобиля по VIN и госномеру;

4. разграничение доступа по ролям;
5. возможность создания сотрудников менеджером;
- оддержка мастеров с административными правами;
7. управление расписанием сотрудников;
- синхронное получение данных через API.
9. Нефункциональные требования:
10. безопасность хранения паролей (хэширование);
- высокая производительность при работе с базой данных;
12. обеспечение целостности данных;
13. масштабируемость архитектуры;
- добовство сопровождения и расширения системы.

Таким образом, сформированные требования определяют основу для проектирования архитектуры серверной логики.

2.2. Проектирование архитектуры backend

Архитектура разрабатываемого серверного приложения базируется на архитектурном паттерне Model-View-Template (MTV), реализованном в веб-фреймворке Django. Данный паттерн обеспечивает разделение ответственности между компонентами системы.

Модель (Model) отвечает за хранение и обработку данных. В разрабатываемой системе модели представлены классами Python, которые отображаются на таблицы в базе данных PostgreSQL. Модели содержат поля, соответствующие атрибутам сущностей, а также методы для работы с данными (например, методы установки и проверки пароля). Дополнительно в системе реализованы модели ролей пользователей, а также специализированные сущности для сотрудников (мастера и менеджеры) и расписания их работы.

Представление (View) содержит бизнес-логику приложения. Представления обрабатывают входящие HTTP-запросы, взаимодействуют с

моделями для получения или сохранения данных, формируют контекст для шаблонов и возвращают HTTP-ответы. В разработанной системе реализованы следующие представления:

- `orders_page` – обработка запросов к главной странице, включая создание новых заявок (POST) и отображение списка существующих заявок (GET) с поддержкой фильтрации по статусу.
- `edit_order` – обработка запросов на редактирование существующей заявки.
- `get_client_by_phone` – API-эндпоинт для получения данных клиента по номеру телефона (используется для автозаполнения формы).
- `get_car_by_plate` – API-эндпоинт для получения данных автомобиля по государственному регистрационному номеру.
- `check_plate_owner` – API-эндпоинт для проверки принадлежности автомобиля указанному клиенту.
- `register_view` и `login_view` – обработка регистрации и авторизации пользователей.
- `register_staff_view` – создание сотрудников (мастеров и менеджеров) с учётом выбранной роли и дополнительных параметров (например, признак администратора для мастера).
- `schedule_page` – управление расписанием сотрудников, включая добавление смен и их просмотр.
- `repair_page` – отображение информации о процессе ремонта и диагностических данных.

Шаблон (Template) отвечает за формирование пользовательского интерфейса. Шаблоны представляют собой HTML-файлы с вкраплениями специального шаблонного языка Django, позволяющего динамически

подставлять данные из контекста, формировать циклы и условные конструкции. В системе реализованы модальные формы регистрации и авторизации, а также интерфейсы управления заявками и расписанием.

Взаимодействие компонентов осуществляется по следующей схеме:

пользователь инициирует HTTP-запрос (GET или POST) к определённому URL-адресу.

система маршрутизации (URL dispatcher) сопоставляет запрошенный URL с соответствующим представлением.

представление выполняет необходимую бизнес-логику: валидацию входных данных, взаимодействие с моделями (чтение или запись в базу данных).

при необходимости представление формирует контекст и передаёт его в шаблон для рендеринга.

сформированный HTML- или JSON-ответ возвращается пользователю.

Дополнительно в системе реализован механизм разграничения доступа на основе ролей пользователей. Проверка прав доступа осуществляется на уровне представлений с использованием пользовательского декоратора, что позволяет ограничивать выполнение операций в зависимости от роли (клиент, мастер, менеджер, администратор).

Для обеспечения целостности данных при выполнении операций, затрагивающих несколько таблиц базы данных, используется механизм транзакций. В разработанной системе транзакции применяются при создании заявки и регистрации пользователей, поскольку данные операции могут включать создание записей сразу в нескольких таблицах (users, client, cars, orders, а также таблицах сотрудников). В случае возникновения ошибки на любом из этапов выполнения транзакции все внесённые изменения откатываются, что гарантирует отсутствие частично сохранённых данных.

В Django по умолчанию используется уровень изоляции READ COMMITTED (для PostgreSQL). Этого достаточно для предотвращения грязного чтения и обеспечения согласованности данных в рамках одной заявки. Переход на REPEATABLE READ не требуется, так как конфликты параллельного создания заявок на один автомобиль маловероятны и устраняются за счёт уникальных ограничений базы данных.

Аутентификация пользователей в системе реализована на основе сессий с сохранением идентификатора пользователя и его роли в серверной сессии. Это позволяет идентифицировать пользователя при последующих запросах и применять соответствующие ограничения доступа. Архитектура системы предусматривает возможность дальнейшего расширения механизма аутентификации (например, внедрение JWT или интеграции с внешними сервисами) без изменения основной бизнес-логики.

2.3. Проектирование BPMN-диаграммы бизнес-процессов

Проектирование серверного приложения требует формализации ключевых бизнес-процессов системы. Для этой цели используется нотация BPMN 2.0 (Business Process Model and Notation), позволяющая описать последовательность действий пользователей и системы в виде формализованных диаграмм.

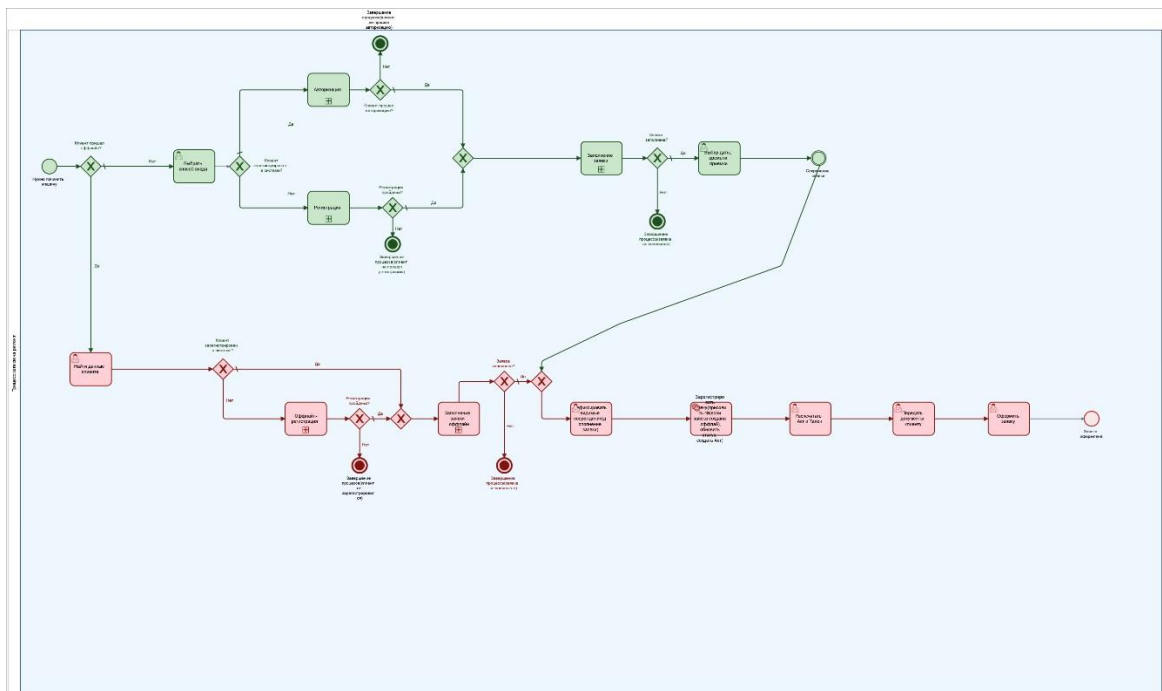


Рис.1. BPMN-диаграмма процесса «Запись на ремонт»

Процесс декомпозирован на два независимых потока выполнения с использованием переиспользуемых подпроцессов, и включает следующие логические этапы:

1. Маршрутизация и идентификация клиента: В начале процесса эксклюзивный шлюз (XOR) определяет формат обращения;
 - для оффлайн-потока: администратор осуществляет поиск данных клиента в системе. Если клиент новый, запускается подпроцесс «Оффлайн-регистрация» (рис.2);

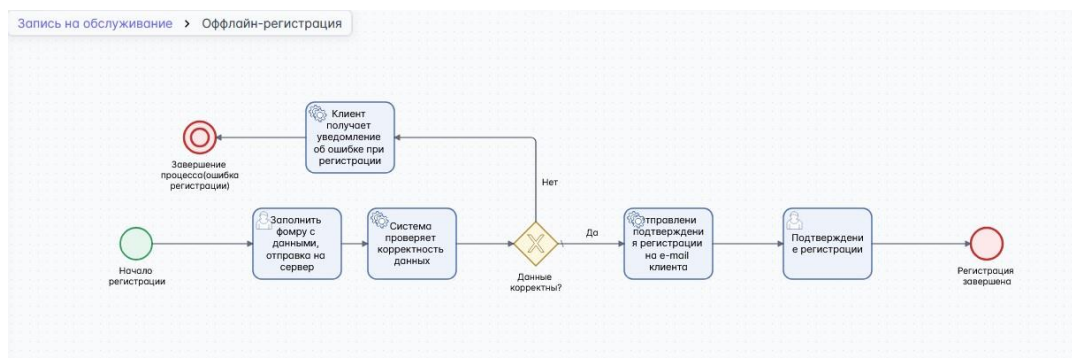


Рис.2. Подпроцесс «Оффлайн-регистрация»

– для онлайн-потока: клиент самостоятельно выбирает способ входа.

Предусмотрены подпроцессы «Авторизация» (рис.3) и «Регистрация» (рис.4). Внутри подпроцессов аутентификации система проводит валидацию введенных данных на сервере. В случае регистрации обязательно реализован шаг отправки подтверждения на e-mail клиента, обеспечивая верификацию контактных данных в базе;



Рис. 3. Подпроцесс «Авторизация»

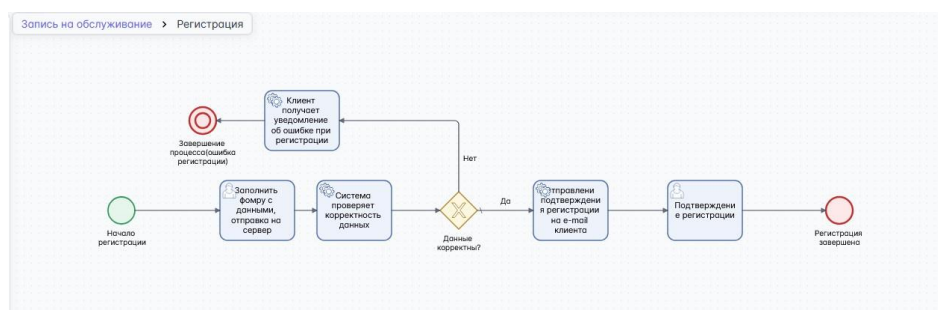


Рис. 4. Подпроцесс «Регистрация»

2. Формирование заявки и проверка уникальности: после успешной идентификации, независимо от канала обращения, инициируется подпроцесс «Заполнение заявки» (рис.5) (или его оффлайн-аналог (рис.6)). На этом этапе в систему вносятся данные об автомобиле (марка, модель, год) и его государственный номер. Важнейшим системным шагом является автоматическая проверка: «Автомобиль уже есть в ремонте?». Если ТС с данным номером уже числится в активных заказах, система блокирует транзакцию и уведомляет о дубликате. Это строго предотвращает появление дублирующих записей и нарушение целостности БД. Только при отрицательном результате проверки вводится описание проблемы.

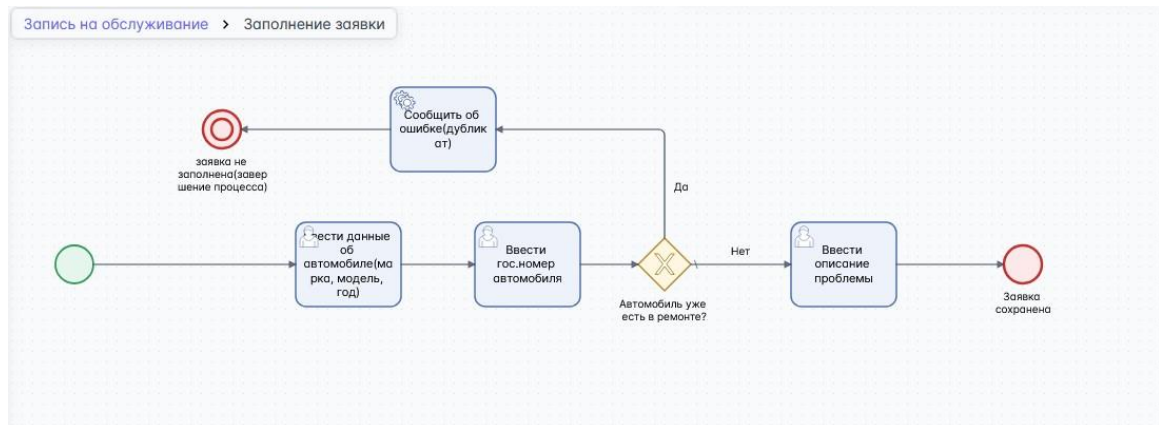


Рис. 5. Подпроцесс «Заполнение заявки»

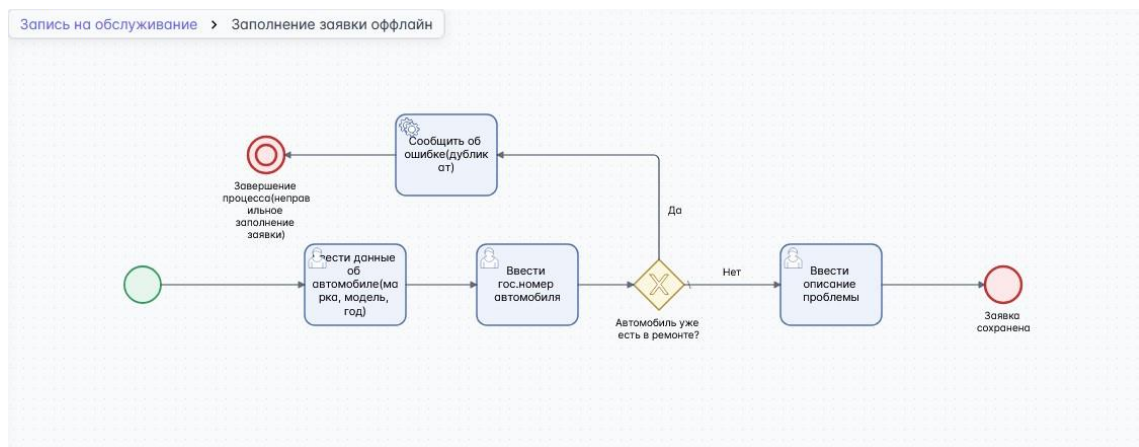


Рис. 6 Подпроцесс «Заполнение заявки оффлайн»

3. Специфичная обработка и завершение (офлайн-поток): при личном визите клиента процесс расширяется дополнительными физическими и системными действиями администратора: производится фиксация видимых повреждений (дополнение данных заявки), после чего в системе регистрируется статус заказа и автоматически формируются печатные документы. Завершающим этапом является распечатка Акта и Талона с последующей передачей их клиенту.

4. Планирование и сохранение (Онлайн-поток): Для пользователей, проходящих процесс удаленно, логика завершается выбором желаемой даты и времени приемки в автосервис. Финальным системным действием является сохранение сформированной заявки в базу данных.

Спроектированная модель выполняет роль комплексной «карты транзакций». Использование BPMN с выделенными подпроцессами и шлюзами проверок позволяет минимизировать «человеческий фактор» — система аппаратно исключает создание нескольких заявок на один автомобиль и жестко регламентирует последовательность действий как пользователя, так и администратора, обеспечивая 100% корректность наполнения таблиц базы данных.

2.4. Реализация моделей данных

Реализация моделей данных осуществлялась с учётом необходимости интеграции с существующей базой данных PostgreSQL. Поскольку структура таблиц уже была спроектирована и создана ранее, модели Django настраивались таким образом, чтобы фреймворк не выполнял автоматическое управление схемой базы данных. Для этого в каждой модели был установлен атрибут `managed = False`, что позволяет использовать Django исключительно как инструмент ORM без вмешательства в структуру таблиц.

```
class User(models.Model):
```

```

id = models.AutoField(primary_key=True)

name = models.CharField(max_length=50)

surname = models.CharField(max_length=50)

email = models.CharField(max_length=100, unique=True)

hash_passwd = models.TextField()

role_id = models.ForeignKey('Role', on_delete=models.DO_NOTHING,
db_column='role_id')

class Meta:

    db_table = 'users'

    managed = False

def set_password(self, raw_password):

    self.hash_passwd = make_password(raw_password)

def check_password(self, raw_password):

    return check_password(raw_password, self.hash_passwd)

```

Модель User отображается на таблицу users и является базовой сущностью системы. Поле email имеет ограничение уникальности. Методы set_password и check_password обеспечивают безопасное хранение паролей. Важно отметить, что для хранения пароля используется поле TextField с ручным применением функции make_password. Такой подход обусловлен необходимостью совместимости с существующей схемой базы данных, однако в промышленной разработке предпочтительнее использовать встроенную систему аутентификации Django. Дополнительно в модель включено поле role_id, являющееся внешним ключом на таблицу ролей, что обеспечивает централизованное управление правами доступа.

```
class Client(models.Model):
```

```
    user = models.OneToOneField(User, on_delete=models.CASCADE,
```

```
        primary_key=True, db_column='user_id')
```

```
    phone = models.CharField(max_length=20)
```

Модель Client связана с User отношением «один к одному» через поле user. Выбор OneToOneField обусловлен тем, что каждому клиенту соответствует ровно один пользователь системы. Такая структура позволяет хранить специфические данные клиента отдельно от общей информации пользователя.

```
class Car(models.Model):
```

```
    id = models.AutoField(primary_key=True)
```

```
    client = models.ForeignKey(User, on_delete=models.CASCADE,
```

```
        db_column='client_id')
```

```
    brand = models.CharField(max_length=100)
```

```
    model = models.CharField(max_length=100)
```

```
    vin = models.CharField(max_length=50, unique=True, null=True, blank=True)
```

```
    plate_number = models.CharField(max_length=20, unique=True)
```

```
    year_produced = models.IntegerField(null=True, blank=True)
```

```
    color = models.CharField(max_length=50, null=True, blank=True)
```

Модель Car содержит информацию об автомобиле и связана с пользователем через внешний ключ. Наиболее значимым архитектурным решением стало использование ForeignKey вместо IntegerField для хранения идентификатора владельца. Это позволило использовать преимущества ORM,

включая методы оптимизации запросов (`select_related`), автоматическую проверку ссылочной целостности и каскадное удаление.

```
class Order(models.Model):  
  
    id = models.AutoField(primary_key=True)  
  
    status_id = models.IntegerField(null=True, blank=True)  
  
    problem_desc = models.TextField(blank=True, null=True)  
  
    created_at = models.DateTimeField(auto_now_add=True)  
  
    car = models.ForeignKey(Car, on_delete=models.CASCADE,  
                            db_column='car_id', null=True, blank=True)  
  
    client = models.ForeignKey(Client, on_delete=models.CASCADE,  
                               db_column='client_id')
```

В модели `Order` поля `car` и `client` реализованы как внешние ключи, что обеспечивает корректное связывание данных между таблицами. Это позволяет использовать оптимизированные запросы, такие как `select_related('car', 'client__user')`, и устраняет проблему множественных обращений к базе данных.

Дополнительно в системе реализованы модели сотрудников:

`Master` — хранит данные мастеров, включая признак административных прав (`is_admin`);

`Manager` — хранит данные менеджеров;

`Role` — справочник ролей пользователей.

Такое разделение позволяет гибко управлять доступом и логикой системы.

Для наглядного представления структуры базы данных была разработана ER-диаграмма. Она отражает основные сущности системы, их атрибуты и связи между таблицами. На диаграмме представлены таблицы пользователей, ролей, клиентов, мастеров, менеджеров, автомобилей, заявок, расписания, а также справочные таблицы, необходимые для поддержки бизнес-логики автосервиса.

ER-диаграмма позволяет визуально определить связи между ключевыми объектами системы. Например, пользователь связан с ролью, клиент является расширением пользователя, автомобиль привязан к клиенту, а заявка связывает клиента, автомобиль и статус выполнения. Отдельно выделены сущности сотрудников — мастер и менеджер, а также таблица расписания, обеспечивающая планирование рабочих смен.

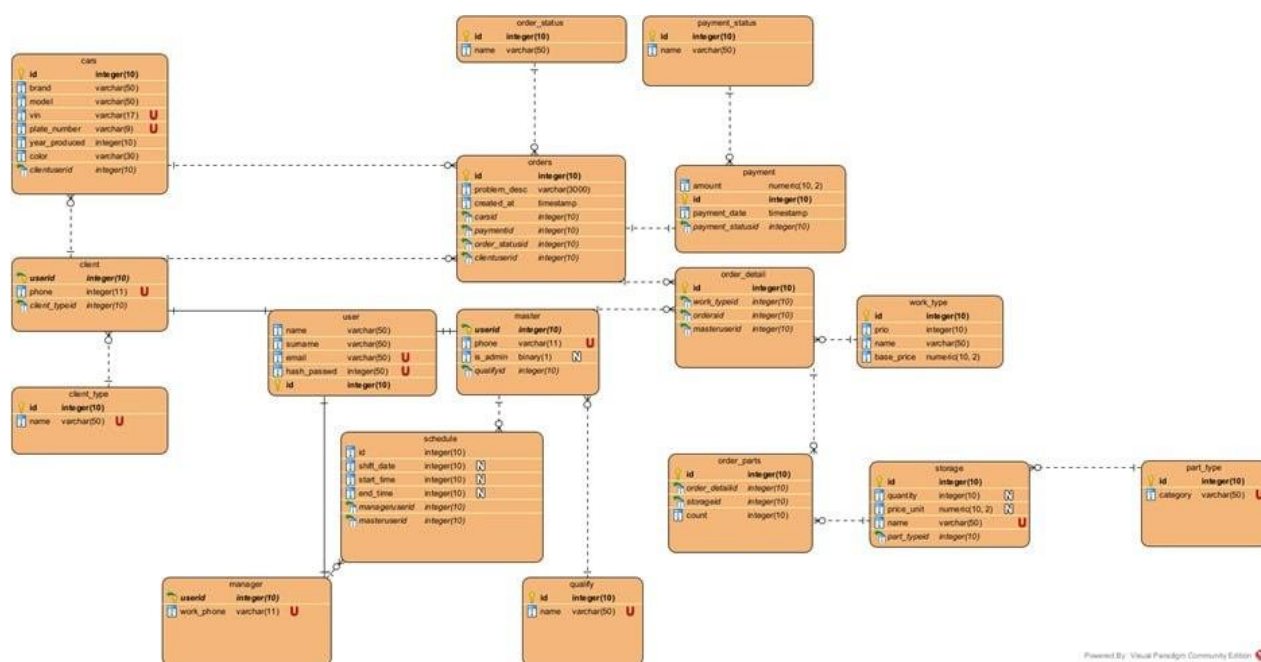


Рис. 7 ER-диаграмма структуры данных

URL Pattern	View функция	Назначение
/	orders_page (GET)	Главная страница
/	orders_page (POST)	Создание новой заявки
/edit/<id>/	edit_order	Редактирование заявки
/get-client-by-phone/	get_client_by_phone	API автозаполнения клиента
/get-car-by-plate/	get_car_by_plate	API автозаполнения авто
/check-plate-owner/	check_plate_owner	API проверки владельца
/schedule/	schedule_page	Управление расписанием
/register/	register_view	Регистрация пользователя
/register_staff/	register_staff_view	Создание сотрудников

Для наглядного представления потока обработки запроса приведена диаграмма последовательности для основного сценария «Создание заявки».

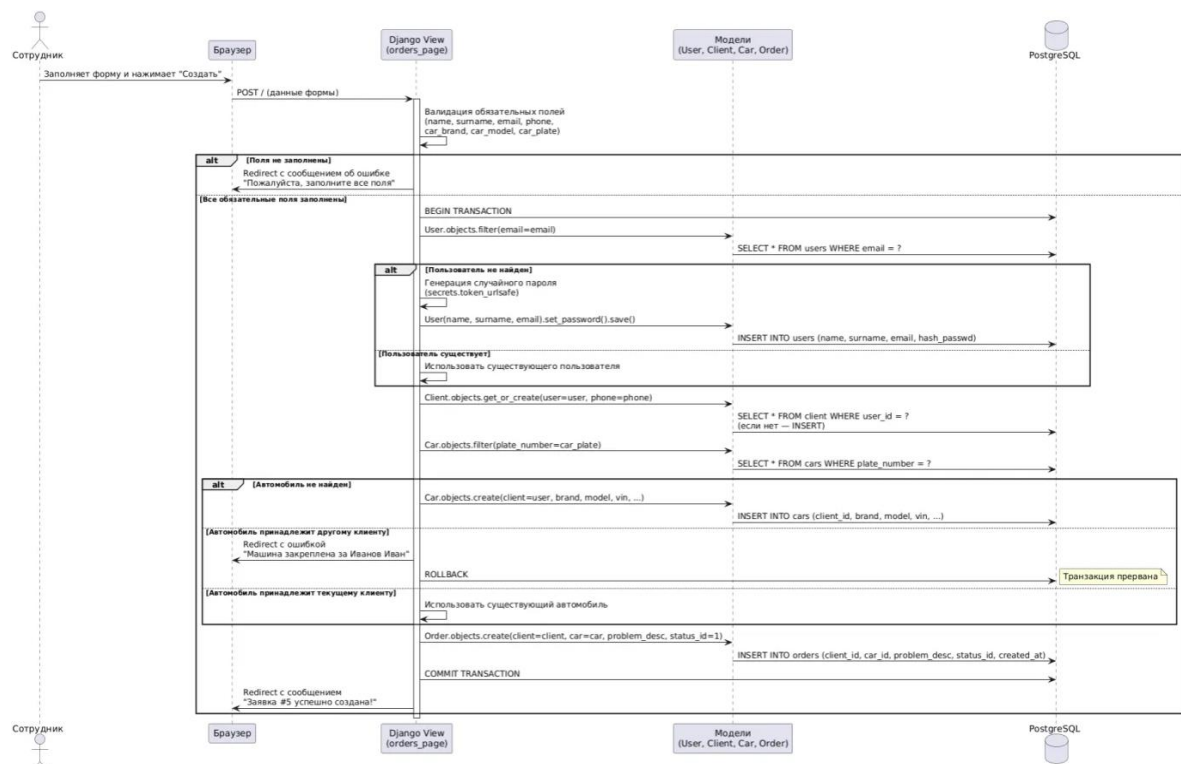


Рис. 7 Диаграмма последовательности

В рамках данного сценария пользователь инициирует отправку формы, после чего запрос поступает в представление `orders_page`. Далее выполняется проверка данных, создание или поиск пользователя и клиента, проверка автомобиля и создание заявки. Все операции выполняются в рамках транзакции, после чего пользователю возвращается результат выполнения.

Диаграмма иллюстрирует взаимодействие между компонентами системы: клиентом, серверной логикой (View), моделями и базой данных. Это позволяет визуально оценить последовательность операций и выявить потенциальные узкие места.

2.5 Реализация бизнес-логики

Реализация бизнес-логики приложения выполнена на уровне представлений (views) и охватывает основные сценарии взаимодействия пользователей с системой. Логика построена с учётом разграничения прав доступа, обеспечения целостности данных и поддержки работы с существующей структурой базы данных.

Центральным элементом является представление `orders_page`, которое обрабатывает как GET-, так и POST-запросы. При обработке GET-запроса выполняется получение списка заявок с возможностью фильтрации по статусу. Для этого используется параметр `status`, передаваемый в URL. При обработке POST-запроса реализуется создание новой заявки.

Процесс создания заявки включает несколько последовательных этапов:

олучение данных из формы и проверка заполненности обязательных полей.

роверка роли пользователя: если пользователь является клиентом, его персональные данные (имя, фамилия, email, телефон) автоматически подставляются из сессии и не могут быть изменены вручную.

поиск пользователя по email. В случае отсутствия пользователя создаётся новая запись с ролью клиента, полученной из таблицы ролей.

создание или обновление записи клиента с актуальным номером телефона.

проверка существования автомобиля по государственному номеру.

создание новой заявки с привязкой к клиенту и автомобилю и установкой начального статуса.

Все перечисленные операции выполняются в рамках транзакции, что гарантирует атомарность и предотвращает появление несогласованных данных.

Представление `edit_order` реализует редактирование существующих заявок. Логика доступа построена с учётом ролей пользователей:

клиент может редактировать только описание проблемы;

менеджер и администратор могут изменять статус заявки;

пользователь не имеет доступа к заявкам, не принадлежащим ему.

Проверка доступа осуществляется на основе данных, сохранённых в сессии пользователя.

Для повышения удобства работы с системой реализованы API-эндпоинты, обеспечивающие асинхронное взаимодействие с сервером:

`get`

`list`

`create`

`update`

`delete`

`check`

`login`

Данные эндпоинты возвращают ответы в формате JSON и используются в клиентской части через JavaScript без перезагрузки страницы.

`logout`

`get_plate` — возвращает информацию об автомобиле по государственному номеру;

`get_client` — возвращает данные клиента по номеру телефона, что

Отдельным направлением является реализация управления пользователями. Представление `register_staff_view` позволяет менеджеру или администратору создавать новых сотрудников. В зависимости от выбранной роли автоматически создаются соответствующие записи в таблицах `Master` или `Manager`. Для мастеров предусмотрена возможность установки признака административных прав (`is_admin`), что расширяет их функциональные возможности.

Функциональность расписания реализована в представлении `schedule_page`. Менеджеры и администраторы могут добавлять новые смены сотрудников, указывая дату и временной интервал. При этом выполняется проверка прав доступа: мастера имеют возможность только просматривать расписание, но не могут изменять его. Смены сохраняются в таблице `ScheduleShift` и могут иметь различные статусы, например «черновик» или «опубликовано», что позволяет разделить этапы планирования и утверждения расписания.

Для оптимизации работы с базой данных используется метод `select_related`, позволяющий загружать связанные объекты одним SQL-запросом. Это существенно снижает количество обращений к базе данных и повышает производительность системы при работе со списками заявок.

Обработка ошибок реализована с использованием конструкции `try/except`. Ошибки, связанные с нарушением ограничений базы данных (`IntegrityError`), обрабатываются отдельно, что позволяет формировать понятные сообщения для пользователя (например, при попытке создать пользователя с уже существующим email или автомобиль с дублирующимся VIN). Общие исключения перехватываются для предотвращения аварийного завершения работы приложения.

Разграничение доступа реализовано с использованием пользовательского декоратора `role_required`, который проверяет роль

пользователя перед выполнением представления. Это позволяет централизованно управлять правами доступа и упрощает сопровождение системы.

Таким образом, реализованная бизнес-логика обеспечивает корректную обработку пользовательских сценариев, учитывает роли пользователей, поддерживает целостность данных и обеспечивает возможность дальнейшего масштабирования системы.

2.6 Тестирование серверной логики

Тестирование разработанного программного обеспечения проводилось в соответствии с методикой, определённой в теоретической части работы. Испытаниям подвергались как отдельные компоненты системы, так и сквозные пользовательские сценарии, включающие взаимодействие с базой данных PostgreSQL, обработку HTTP-запросов, проверку прав доступа и выполнение операций с несколькими связанными таблицами.

Основное внимание при тестировании уделялось следующим аспектам:

- корректность регистрации пользователей;
- создание заявок на ремонт;
- проверка принадлежности автомобиля клиенту;
- разграничение прав доступа между клиентами, мастерами, менеджерами и администраторами;
- корректность работы расписания сотрудников;
- сохранение целостности данных при ошибках;
- корректность работы API-эндпоинтов автозаполнения.

В таблице 1 представлены результаты функционального тестирования серверной логики.

Таблица 1

№	Сценарий	Выполняемое действие	Ожидаемый результат	Фактический результат
1	Регистрация нового клиента	POST-запрос на /register/ с корректными данными клиента	Создаются записи в таблицах users и client, пользователь авторизуется	Соответствует ожидаемому
2	Авторизация пользователя	POST-запрос на /login/ с корректными email и паролем	В сессию сохраняются user_id, role_id, user_role, имя и фамилия пользователя	Соответствует ожидаемому
3	Создание новой заявки клиентом	POST-запрос на /orders/ с данными автомобиля и описанием проблемы	Создаётся заявка в таблице orders, клиент видит её в списке своих заявок	Соответствует ожидаемому
4	Создание заявки на чужой автомобиль	POST-запрос на /orders/ с госномером автомобиля, принадлежащего другому пользователю	Создание заявки блокируется, выводится сообщение об ошибке	Соответствует ожидаемому

5	Фильтрация заявок по статусу	GET-запрос /orders/?status=1	Отображаются только заявки со статусом «Новая»	Соответствует ожидаемому
6	Редактирование заявки менеджером	POST-запрос на /edit/<id>/	Описание и статус заявки обновляются	Соответствует ожидаемому
7	Запрет изменения статуса клиентом	POST-запрос клиента на /edit/<id>/ с изменением статуса	Изменяется только описание проблемы	Соответствует ожидаемому
8	Автозаполнение клиента по телефону	GET-запрос /get-client-by-phone/?phone=...	Возвращается JSON с данными клиента	Соответствует ожидаемому
9	Создание смены в расписании менеджером	POST-запрос на /schedule/ с мастером, датой и временем смены	Создаётся запись в таблице schedule_shifts	Соответствует ожидаемому
10	Запрет редактирования расписания мастером	POST-запрос мастера на /schedule/ с попыткой добавить смену	Операция блокируется, сообщение об отсутствии прав	Соответствует ожидаемому

В ходе тестирования было подтверждено, что серверная логика корректно обрабатывает основные сценарии работы системы. Особое значение имеет проверка разграничения прав доступа: клиент видит только собственные заявки, мастер имеет доступ к расписанию и странице ремонта без возможности редактировать расписание, а менеджер получает доступ к функциям управления заявками, сотрудниками и сменами.

Также была проверена работа транзакций. При создании заявки операция затрагивает несколько таблиц: users, client, cars и orders. В случае возникновения ошибки на любом из этапов изменения откатываются, что предотвращает появление неполных или некорректных записей. Это подтверждает соблюдение требований к целостности данных.

Дополнительно проверена работа API-эндпоинтов, возвращающих данные в формате JSON. Они обеспечивают автозаполнение форм и повышают удобство работы пользователя, не требуя перезагрузки страницы.

Таким образом, результаты тестирования подтверждают корректность реализации серверной логики, соответствие функциональным требованиям и готовность системы к дальнейшему расширению.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была спроектирована и реализована серверная логика для системы управления бизнес-процессами автомастерской. Разработанное решение обеспечивает автоматизацию ключевых операций: регистрацию пользователей, создание и обработку заявок на ремонт, учёт автомобилей, управление сотрудниками, разграничение прав доступа и ведение расписания работы мастеров.

В процессе разработки были достигнуты следующие результаты:

проведён анализ предметной области и определены основные бизнес-процессы автомастерской, подлежащие автоматизации.

формулированы функциональные и нефункциональные требования к серверной части информационной системы.

проектирована архитектура backend-приложения на основе паттерна

реализованы модели данных для работы с базой данных PostgreSQL, включая пользователей, роли, клиентов, мастеров, менеджеров, автомобили, заявки и расписание смен.

ведена ролевая модель доступа на основе справочника roles, что позволило разграничить возможности клиентов, мастеров, менеджеров и администраторов.

разработана бизнес-логика создания и редактирования заявок, включая проверку принадлежности автомобиля, обработку статусов и фильтрацию заявок.

реализована регистрация пользователей и сотрудников с автоматическим созданием записей в соответствующих таблицах.

реализован модуль расписания, позволяющий менеджерам и администраторам создавать, удалять и публиковать смены мастеров.

реализованы API-эндпоинты для автозаполнения данных клиента и автомобиля.

проведено функциональное тестирование серверной логики, подтвердившее корректность работы основных сценариев.

Практическая значимость работы заключается в создании программного модуля, который может быть использован для автоматизации деятельности автомастерской. Система обеспечивает централизованное хранение данных, контроль доступа, снижение вероятности ошибок при оформлении заявок и повышение удобства работы сотрудников.

Разработанная архитектура обладает потенциалом дальнейшего развития. В перспективе возможно расширение функциональности за счёт добавления складского учёта, модуля расчёта стоимости ремонта, интеграции с платёжными сервисами, уведомлений по email или SMS, а также вынесения бизнес-логики в отдельный сервисный слой для повышения сопровождаемости кода.

Список использованных источников

- ОСТ 19.301-79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. – Введ. 1980-01-01. – М.: Стандартинформ, 2010. – 4 с.
- Кейт К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – М.: Вильямс, 2018. – 1328 с.
- Долганова О. И. Бизнес-процессы: анализ, моделирование, технологии совершенствования / О. И. Долганова. – М.: КноРус, 2025. – 336 с.
- Лутц М. PostgreSQL. Основы / М. Лутц. – СПб.: БХВ-Петербург, 2022. – 416 с.
- Стружкин Н. П. Базы данных: проектирование: учебник для вузов / Н. П. Стружкин, В. В. Годин. – М.: Издательство Юрайт, 2016. – 477 с.
- Официальная документация Django [Электронный ресурс]. – Режим доступа: <https://docs.djangoproject.com/> – Загл. с экрана.
- Официальная документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> – Загл. с экрана.

Приложение А. Листинги программного кода

```
# ===== models.py =====

from django.db import models

from django.contrib.auth.hashers import make_password, check_password


class Role(models.Model):

    id = models.AutoField(primary_key=True)

    name = models.CharField(max_length=50, unique=True)

    title = models.CharField(max_length=100)


    class Meta:

        db_table = 'roles'

        managed = False


    def __str__(self):

        return self.title


class User(models.Model):

    id = models.AutoField(primary_key=True)

    name = models.CharField(max_length=50)

    surname = models.CharField(max_length=50)
```

```
email = models.CharField(max_length=100, unique=True)
```

```
hash_passwd = models.TextField()
```

```
role_id = models.ForeignKey(  
    Role,  
    on_delete=models.PROTECT,  
    db_column='role_id'  
)
```

```
class Meta:
```

```
    db_table = 'users'
```

```
    managed = False
```

```
def set_password(self, raw_password):
```

```
    self.hash_passwd = make_password(raw_password)
```

```
def check_password(self, raw_password):
```

```
    return check_password(raw_password, self.hash_passwd)
```

```
@property
```

```
def role_name(self):
```

```
return self.role_id.name
```

```
@property
```

```
def is_authenticated(self):
```

```
    return True
```

```
class Client(models.Model):
```

```
    user_id = models.OneToOneField(User, on_delete=models.CASCADE,  
db_column='user_id', primary_key=True)
```

```
    phone = models.CharField(max_length=20)
```

```
    type_client_id = models.IntegerField(null=True, blank=True)
```

```
class Meta:
```

```
    db_table = 'client'
```

```
    managed = False
```

```
class Car(models.Model):
```

```
    id = models.AutoField(primary_key=True)
```

```
    client_id = models.IntegerField()
```

```
brand = models.CharField(max_length=100)

model = models.CharField(max_length=100)

vin = models.CharField(max_length=50, null=True, blank=True)

plate_number = models.CharField(max_length=20)

year_produced = models.IntegerField(null=True, blank=True)

color = models.CharField(max_length=50, null=True, blank=True)
```

```
class Meta:
```

```
    db_table = 'cars'
```

```
    managed = False
```

```
def __str__(self):
```

```
    return f"{self.brand} {self.model} ({self.plate_number})"
```

```
class Order(models.Model):
```

```
    id = models.AutoField(primary_key=True)
```

```
    status_id = models.IntegerField(null=True, blank=True)
```

```
    payment_id = models.IntegerField(null=True, blank=True)
```

```
    problem_desc = models.TextField(blank=True, null=True)
```

```
    created_at = models.DateTimeField(auto_now_add=True)
```



```
car_id = models.ForeignKey(
    Car,
    on_delete=models.CASCADE,
    db_column='car_id'
)
```

```
client_id = models.ForeignKey(
    Client,
    on_delete=models.CASCADE,
    db_column='client_id'
)
```

```
class Meta:
    db_table = 'orders'
    managed = False
```

```
class ScheduleShift(models.Model):
    STATUS_CHOICES = [
        ('draft', 'Черновик'),
        ('published', 'Опубликовано'),
```

```
]
```

```
id = models.AutoField(primary_key=True)
```

```
employee = models.ForeignKey(
```

```
    User,
```

```
    on_delete=models.CASCADE,
```

```
    db_column='employee_id'
```

```
)
```

```
date = models.DateField()
```

```
start_time = models.TimeField()
```

```
end_time = models.TimeField()
```

```
status = models.CharField(
```

```
    max_length=20,
```

```
    choices=STATUS_CHOICES,
```

```
    default='draft'
```

```
)
```

```
class Meta:
```

```
    db_table = 'schedule_shifts'
```

```
def __str__(self):
```

```
return f'{self.employee.surname} {self.employee.name}: {self.date}'
```

```
class Manager(models.Model):
```

```
    user_id = models.OneToOneField(
```

```
        User,
```

```
        on_delete=models.CASCADE,
```

```
        db_column='user_id',
```

```
        primary_key=True
```

```
    )
```

```
    work_phone = models.CharField(max_length=30, blank=True, null=True)
```

```
class Meta:
```

```
    db_table = 'manager'
```

```
    managed = False
```

```
class Master(models.Model):
```

```
    user_id = models.OneToOneField(
```

```
        User,
```

```
        on_delete=models.CASCADE,
```

```
        db_column='user_id',
```

```

        primary_key=True
    )

    qualify_id = models.IntegerField(null=True, blank=True)

    phone = models.CharField(max_length=30, blank=True, null=True)

    is_admin = models.BooleanField(default=False)


class Meta:

    db_table = 'master'

    managed = False

# ===== views.py =====

from django.shortcuts import render, redirect

from django.contrib import messages

from django.db import transaction, IntegrityError

from django.http import JsonResponse

from django.views.decorators.csrf import csrf_exempt

from datetime import datetime, timedelta

import json

import secrets


from .models import Client, Car, Order, User, ScheduleShift, Role, Manager,
Master

```

```
from .decorators import role_required
```

```
def set_user_session(request, user):
```

```
    request.session["user_id"] = user.id
```

```
    request.session["role_id"] = user.role_id.id
```

```
    request.session["user_role"] = user.role_id.name
```

```
    request.session["user_name"] = user.name
```

```
    request.session["user_surname"] = user.surname
```

```
def home_page(request):
```

```
    return render(request, 'base.html')
```

```
@role_required('client', 'manager', 'admin')
```

```
def orders_page(request):
```

```
    status_filter = request.GET.get('status', 'all')
```

```
    user_role = request.session.get("user_role")
```

```
    session_user_id = request.session.get("user_id")
```

```

if request.method == "POST":

    name = request.POST.get('name')

    surname = request.POST.get('surname')

    email = request.POST.get('email')

    phone = request.POST.get('phone')

    car_brand = request.POST.get('car_brand')

    car_model = request.POST.get('car_model')

    car_plate = request.POST.get('car_plate')

    vin = request.POST.get('vin', "")

    car_year = request.POST.get('car_year', None)

    car_color = request.POST.get('car_color', "")

    problem_desc = request.POST.get('problem_desc', "")


if user_role == "client":

    current_user =
User.objects.select_related("role_id").get(id=session_user_id)

    current_client = Client.objects.get(user_id=current_user)


    name = current_user.name

    surname = current_user.surname

    email = current_user.email

```

```

phone = current_client.phone

required_fields = [name, surname, email, phone, car_brand, car_model,
car_plate]

if not all(required_fields):

    messages.error(request, 'Пожалуйста, заполните все обязательные поля')

    return redirect('/orders/')

try:

    with transaction.atomic():

        user = User.objects.select_related("role_id").filter(email=email).first()

        if not user:

            client_role = Role.objects.get(name="client")

            user = User(

                name=name,

                surname=surname,

                email=email,

                role_id=client_role

```

)

```
temp_password = secrets.token_urlsafe(8)
```

```
print(f'INFO: Новый пользователь {email}, временный пароль:  
{temp_password}')
```

```
user.set_password(temp_password)
```

```
user.save()
```

```
messages.success(request, f'Новый пользователь {surname} {name}  
создан!')
```

```
else:
```

```
messages.info(request, f'Добро пожаловать, {user.surname}  
{user.name}!')
```

```
client, created = Client.objects.get_or_create(
```

```
    user_id=user,
```

```
    defaults={'phone': phone}
```

```
)
```

```
if not created and client.phone != phone:
```

```
    client.phone = phone
```



```

client.save()

existing_car = Car.objects.filter(plate_number=car_plate.upper()).first()

if existing_car:

    if existing_car.client_id == user.id:

        car = existing_car

        messages.info(request, f'Машина {car_plate} уже есть в системе')

    else:

        owner = User.objects.filter(id=existing_car.client_id).first()

        owner_name = f'{owner.surname} {owner.name}' if owner else
'другим клиентом'

        messages.error(request, f'Машина с госномером {car_plate}
закреплена за {owner_name}')

        return redirect('/orders/')

else:

    car = Car.objects.create(

        client_id=user.id,

        brand=car_brand,

        model=car_model,

        vin=vin if vin else None,

        plate_number=car_plate.upper(),

```

```
        year_produced=car_year if car_year else None,  
        color=car_color if car_color else None  
    )
```

```
order = Order.objects.create(  
    client_id=client,  
    car_id=car,  
    problem_desc=problem_desc,  
    status_id=1  
)
```

```
messages.success(request, f'Заявка #{order.id} успешно создана!')  
return redirect('/orders/')
```

```
except IntegrityError as e:
```

```
    messages.error(request, f'Ошибка целостности данных: {e}')  
    return redirect('/orders/')
```

```
except Exception as e:
```

```
    messages.error(request, f'Неизвестная ошибка: {e}')  
    return redirect('/orders/')
```

```

orders_all = Order.objects.select_related(

    'client_id',

    'client_id__user_id',

    'car_id'

).order_by('-created_at')


if user_role == "client":

    orders_all = orders_all.filter(client_id__user_id=session_user_id)


if status_filter != 'all':

    orders = orders_all.filter(status_id=status_filter)

else:

    orders = orders_all


orders_new = orders_all.filter(status_id=1)

orders_work = orders_all.filter(status_id=2)

orders_done = orders_all.filter(status_id=3)


client_phone = ""

client_email = ""

```

```
if session_user_id:

    current_user = User.objects.filter(id=session_user_id).first()

    current_client = Client.objects.filter(user_id=current_user).first()


if current_user:

    client_email = current_user.email


if current_client:

    client_phone = current_client.phone


return render(request, 'orders.html', {

    'orders': orders,

    'orders_all': orders_all,

    'statuses': {1: 'Новая', 2: 'В работе', 3: 'Завершена'},

    'current_status': status_filter,

    'orders_new': orders_new,

    'orders_work': orders_work,

    'orders_done': orders_done,

    'client_phone': client_phone,

    'client_email': client_email,

})
```

```

def edit_order(request, order_id):

    order = Order.objects.select_related(

        'client_id',

        'client_id__user_id',

        'car_id'

    ).filter(id=order_id).first()


    if not order:

        messages.error(request, 'Заявка не найдена')

        return redirect('/orders/')


    user_role = request.session.get("user_role")

    user_id = request.session.get("user_id")


    if not user_id:

        messages.error(request, 'Сначала войдите в аккаунт')

        return redirect('/')


    if user_role == "client" and order.client_id.user_id.id != user_id:

```

```
messages.error(request, 'У вас нет доступа к этой заявке')

return redirect('/orders/')


if request.method == "POST":

    problem_desc = request.POST.get('problem_desc', "")

    try:

        with transaction.atomic():

            order.problem_desc = problem_desc

            if user_role in ["manager", "admin"]:

                status_id = request.POST.get('status_id', order.status_id)

                if str(status_id) not in {'1', '2', '3'}:

                    messages.error(request, 'Недопустимый статус заявки')

                    return redirect('/orders/')

                order.status_id = int(status_id)

            order.save()

    messages.success(request, f'Заявка #{order.id} успешно обновлена!')
```

```
return redirect('/orders/')
```

```
except Exception as e:
```

```
    messages.error(request, f'Ошибка при обновлении: {str(e)}')
```

```
    return redirect('/orders/')
```

```
return render(request, 'edit_order.html', {'order': order})
```

```
def get_client_by_phone(request):
```

```
    phone = request.GET.get('phone', '')
```

```
    if phone:
```

```
        client = Client.objects.filter(phone=phone).select_related('user_id').first()
```

```
    if client:
```

```
        return JsonResponse({
```

```
            'found': True,
```

```
            'name': client.user_id.name,
```

```
            'surname': client.user_id.surname,
```

```
            'email': client.user_id.email,
```

```
        'phone': client.phone,  
        'client_id': client.user_id.id  
    })
```

```
    return JsonResponse({'found': False})
```

```
def get_car_by_plate(request):
```

```
    plate = request.GET.get('plate', "").upper()
```

```
    if plate:
```

```
        car = Car.objects.filter(plate_number=plate).first()
```

```
        if car:
```

```
            owner = User.objects.filter(id=car.client_id).first()
```

```
            return JsonResponse({
```

```
                'found': True,
```

```
                'brand': car.brand,
```

```
                'model': car.model,
```

```
                'vin': car.vin or "",
```



```

        'year': car.year_produced,

        'color': car.color or "",

        'owner_client_id': car.client_id,

        'owner_name': f"{owner.surname} {owner.name}" if owner else
'неизвестен'

    })

```

```

return JsonResponse({'found': False})

```

```

def check_plate_owner(request):

```

```

    plate = request.GET.get('plate', "").upper()

```

```

    current_client_id = request.GET.get('client_id', None)

```

```

    if plate:

```

```

        car = Car.objects.filter(plate_number=plate).first()

```

```

        if car:

```

```

            if current_client_id and str(car.client_id) == current_client_id:

```

```

                return JsonResponse({

```

```

                    'is_owner': True,

```

```
        'message': 'Это ваша машина',  
    })
```

```
owner = User.objects.filter(id=car.client_id).first()
```

```
return JsonResponse(  
    'is_owner': False,  
    'owner_name': f"{owner.surname} {owner.name}" if owner else 'другим  
клиентом',  
    'message': f"Машина закреплена за {owner.surname} {owner.name}" if  
owner else 'Машина закреплена за другим клиентом'  
    })
```

```
return JsonResponse({'is_owner': True, 'message': ""})
```

```
@role_required('manager', 'master', 'admin')
```

```
def schedule_page(request):
```

```
    user_role = request.session.get("user_role")
```

```
    if request.method == "POST":
```

```
        if user_role not in ["manager", "admin"]:
```

```

    messages.error(request, "У вас нет прав изменять расписание")

    return redirect("/schedule/")

employee_id = request.POST.get("employee_id")

date = request.POST.get("date")

start_time = request.POST.get("start_time")

end_time = request.POST.get("end_time")

if not employee_id or not date or not start_time or not end_time:

    messages.error(request, "Заполните все поля")

    return redirect("/schedule/")

employee = User.objects.filter(id=employee_id,
role_id__name="master").first()

if not employee:

    messages.error(request, "Мастер не найден")

    return redirect("/schedule/")

ScheduleShift.objects.create(

    employee=employee,

```

```
date=date,  
  
start_time=start_time,  
  
end_time=end_time,  
  
status="draft"  
  
)  
  
messages.success(request, "Смена добавлена")  
  
return redirect(request.META.get("HTTP_REFERER", "/schedule/"))
```

```
period = request.GET.get("period", "week")
```

```
start_param = request.GET.get("start")
```

```
if start_param:
```

```
    start_date = datetime.strptime(start_param, "%Y-%m-%d").date()
```

```
else:
```

```
    today = datetime.now().date()
```

```
    start_date = today - timedelta(days=today.weekday())
```

```
if period == "month":
```

```
    start_date = start_date.replace(day=1)
```

```
    next_month = start_date.replace(day=28) + timedelta(days=4)
```

```

    end_date = next_month.replace(day=1)

    days_count = (end_date - start_date).days

else:

    period = "week"

    end_date = start_date + timedelta(days=7)

    days_count = 7

date_list = [start_date + timedelta(days=i) for i in range(days_count)]


shifts = ScheduleShift.objects.select_related("employee",
"employee__role_id").filter(

    date__gte=start_date,

    date__lt=end_date

).order_by("date", "start_time")


employees = User.objects.filter(

    role_id__name="master"

).order_by("surname", "name")


if period == "month":

    prev_date = (start_date - timedelta(days=1)).replace(day=1)

```

```
next_date = end_date
```

```
else:
```

```
prev_date = start_date - timedelta(days=7)
```

```
next_date = start_date + timedelta(days=7)
```

```
return render(request, "schedule.html", {
```

```
    "employees": employees,
```

```
    "shifts": shifts,
```

```
    "date_list": date_list,
```

```
    "period": period,
```

```
    "start_date": start_date,
```

```
    "prev_date": prev_date.strftime("%Y-%m-%d"),
```

```
    "next_date": next_date.strftime("%Y-%m-%d"),
```

```
    "can_edit_schedule": user_role in ["manager", "admin"],
```

```
})
```

```
@role_required('manager', 'admin', 'master')
```

```
def repair_page(request):
```

```
    order = {
```

```
        'id': 105,
```

```

        'plate': 'A777AA 777',

        'damage_initial': 'Вмятина на заднем крыле'

    }

    report_data = {

        'obd_codes': ['P0300', 'P0420'],

        'interior_problems': ['Порвана обивка сиденья', 'Потертость руля'],

        'component_problems': ['Неисправность катушки зажигания', 'Износ
тормозных колодок']

    }

    return render(request, 'repair.html', {

        'order': order,

        'has_report': True,

        'report': report_data

    })

@csrf_exempt

def register_view(request):

    if request.method != "POST":

```

```
return JsonResponse({"error": "Метод не разрешён"}, status=405)
```

```
data = json.loads(request.body)
```

```
name = data.get("first_name", "").strip()
```

```
surname = data.get("last_name", "").strip()
```

```
phone = data.get("phone", "").strip()
```

```
email = data.get("email", "").strip().lower()
```

```
password = data.get("password", "")
```

```
password2 = data.get("password2", "")
```

```
if not name or not surname or not phone or not email or not password:
```

```
    return JsonResponse({"error": "Заполните все поля"}, status=400)
```

```
if password != password2:
```

```
    return JsonResponse({"error": "Пароли не совпадают"}, status=400)
```

```
if User.objects.filter(email=email).exists():
```

```
    return JsonResponse({"error": "Пользователь уже существует"},  
status=400)
```



```
try:

    with transaction.atomic():

        client_role = Role.objects.get(name="client")

        user = User(

            name=name,

            surname=surname,

            email=email,

            role_id=client_role

        )

        user.set_password(password)

        user.save()

        Client.objects.create(

            user_id=user,

            phone=phone

        )

        set_user_session(request, user)

    return JsonResponse({"success": True})
```

```
except Exception as e:
```

```
    return JsonResponse({"error": str(e)}, status=500)
```

```
@csrf_exempt
```

```
def login_view(request):
```

```
    if request.method != "POST":
```

```
        return JsonResponse({"error": "Метод не разрешён"}, status=405)
```

```
    data = json.loads(request.body)
```

```
    email = data.get("email", "").strip().lower()
```

```
    password = data.get("password", "")
```

```
    user = User.objects.select_related("role_id").filter(email=email).first()
```

```
    if user and user.check_password(password):
```

```
        set_user_session(request, user)
```

```
        return JsonResponse({"success": True})
```

```
return JsonResponse({"error": "Неверная почта или пароль"}, status=400)
```

```
def logout_view(request):
```

```
    request.session.flush()
```

```
    return JsonResponse({"success": True})
```

```
@role_required('manager', 'admin')
```

```
def delete_shift(request, shift_id):
```

```
    shift = ScheduleShift.objects.filter(id=shift_id).first()
```

```
    if shift:
```

```
        shift.delete()
```

```
        messages.success(request, "Смена удалена")
```

```
    return redirect(request.META.get("HTTP_REFERER", "/schedule/"))
```

```
@role_required('manager', 'admin')
```

```
def publish_schedule(request):
```

```
ScheduleShift.objects.filter(status="draft").update(status="published")

messages.success(request, "Расписание опубликовано")

return redirect(request.META.get("HTTP_REFERER", "/schedule/"))
```

```
@csrf_exempt
```

```
@role_required('manager', 'admin')
```

```
def register_staff_view(request):
```

```
    if request.method != "POST":
```

```
        return JsonResponse({"error": "Метод не разрешён"}, status=405)
```

```
    data = json.loads(request.body)
```

```
    name = data.get("first_name", "").strip()
```

```
    surname = data.get("last_name", "").strip()
```

```
    phone = data.get("phone", "").strip()
```

```
    email = data.get("email", "").strip().lower()
```

```
    password = data.get("password", "")
```

```
    role_name = data.get("role", "").strip()
```

```
    allowed_roles = ["master", "manager"]
```

```

if role_name not in allowed_roles:

    return JsonResponse({"error": "Менеджер может создавать только
мастеров и менеджеров"}, status=400)

if not name or not surname or not phone or not email or not password:

    return JsonResponse({"error": "Заполните все поля"}, status=400)

if User.objects.filter(email=email).exists():

    return JsonResponse({"error": "Пользователь уже существует"},
status=400)

try:

    with transaction.atomic():

        role = Role.objects.get(name=role_name)

        user = User(

            name=name,

            surname=surname,

            email=email,

            role_id=role

        )

```

```
user.set_password(password)
```

```
user.save()
```

```
if role_name == "master":
```

```
    raw_is_admin = data.get("is_admin", False)
```

```
    is_admin = raw_is_admin in ["true", "True", "1", 1, True, "on"]
```

```
    Master.objects.create(
```

```
        user_id=user,
```

```
        qualify_id=None,
```

```
        phone=phone,
```

```
        is_admin=is_admin
```

```
    )
```

```
elif role_name == "manager":
```

```
    Manager.objects.create(
```

```
        user_id=user,
```

```
        work_phone=phone
```

```
    )
```

```
return JsonResponse({"success": True})
```

```

except Exception as e:

    return JsonResponse({"error": str(e)}, status=500)

# ===== urls.py =====

from django.contrib import admin

from django.urls import path

from main import views

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", views.home_page, name='home'),

    path('orders/', views.orders_page, name='orders'),

    path('edit/<int:order_id>/', views.edit_order, name='edit_order'),

    path('get-client-by-phone/', views.get_client_by_phone,
name='get_client_by_phone'),

    path('schedule/', views.schedule_page, name='schedule'),

    path('schedule/delete/<int:shift_id>/', views.delete_shift, name='delete_shift'),

    path('schedule/publish/', views.publish_schedule, name='publish_schedule'),

    path('get-car-by-plate/', views.get_car_by_plate, name='get_car_by_plate'),

    path('check-plate-owner/', views.check_plate_owner,
name='check_plate_owner'),

    path('repair/', views.repair_page, name='repair'),

```

```

path('register/', views.register_view),

path('login/', views.login_view),

path('logout/', views.logout_view),

path('register_staff/', views.register_staff_view),

]

# ===== decorators.py =====

from django.shortcuts import redirect

from django.contrib import messages

def role_required(*roles):

    def decorator(view_func):

        def wrapper(request, *args, **kwargs):

            user_role = request.session.get("user_role")

            if not user_role:

                messages.error(request, "Сначала войдите в аккаунт")

                return redirect("/")

            if user_role not in roles:

                messages.error(request, "У вас нет доступа к этой странице")

                return redirect("/")

            return view_func(request, *args, **kwargs)

        return wrapper

    return decorator

```